Amazon EKS
Cluster Upgrade

Standard Operating Procedure

Console-Based Approach with NoExecute Taint Node Drain Strategy

Field	Details
Scope	Amazon EKS Cluster — Control Plane, Node Groups, Add-ons
Upgrade Method	AWS Management Console + kubectl CLI
Node Drain Strategy	NoExecute Taint (kubectl taint) — One node at a time
Version	1.0

PHASE 0 — PRE-UPGRADE PREPARATION

Pre-Upgrade Checklist

Complete all checks below before initiating any upgrade activity. Skipping pre-checks is the leading cause of failed or disruptive upgrades.

> **WARNING:** EKS only supports upgrading one minor version at a time. If your cluster is on 1.26, you must go 1.26 → 1.27 → 1.28. You cannot skip versions.

1.1 Review Release Notes & API Deprecations

- Visit the AWS EKS Kubernetes version documentation for your target version.
- Check for deprecated or removed API versions (e.g., apps/v1beta1 removed in 1.16).
- Run the following command to identify deprecated APIs in use on your cluster:

```
# Install pluto (API deprecation checker)
curl -L
https://github.com/FairwindsOps/pluto/releases/latest/download/pluto_linux_amd64.tar.gz
| tar xz

# Scan your cluster for deprecated APIs
./pluto detect-all-in-cluster --target-versions k8s=v1.29.0

# Check current cluster version
kubectl version --short
aws eks describe-cluster --name <cluster-name> --query 'cluster.version'
```

1.2 Identify All Node Groups & Add-ons

```
# List all managed node groups
aws eks list-nodegroups --cluster-name <cluster-name>

# List all installed add-ons and current versions
aws eks list-addons --cluster-name <cluster-name>

# Check each add-on version
aws eks describe-addon --cluster-name <cluster-name> --addon-name vpc-cni
aws eks describe-addon --cluster-name <cluster-name> --addon-name kube-proxy
aws eks describe-addon --cluster-name <cluster-name> --addon-name coredns
```

> **NOTE:** Verify add-on version compatibility with your target Kubernetes version at: https://docs.aws.amazon.com/eks/latest/userguide/add-ons-configuration.html

1.3 Backup Cluster State

```
# Export all resources across all namespaces
kubectl get all -A -o yaml > cluster-backup-$(date +%Y%m%d).yaml

# Export configmaps and secrets
kubectl get configmaps -A -o yaml > configmaps-backup.yaml
kubectl get pvc -A -o yaml > pvc-backup.yaml

# Check PodDisruptionBudgets (important before draining)
kubectl get pdb -A
```

1.4 Verify Cluster Health

```
# All nodes should be in Ready state
kubectl get nodes

# No pods should be in CrashLoopBackOff or Error state
kubectl get pods -A | grep -vE 'Running|Completed'

# Check node resource availability
kubectl top nodes
```

PHASE 1 — CONTROL PLANE UPGRADE (AWS CONSOLE)

Upgrading the EKS Control Plane

The control plane upgrade is performed entirely through the AWS Management Console. AWS manages the upgrade with zero downtime to running workloads — your pods continue running throughout this phase.

1 **Navigate to the EKS Console**
Open AWS Console → Services → EKS → Clusters → Select your cluster.

2 **Initiate the Upgrade**
On the cluster overview page, locate the Kubernetes version field. Click the 'Upgrade now' button next to it.

3 **Select Target Version**
Choose the next minor version (e.g., 1.28 → 1.29). Review the confirmation dialog carefully.

4 **Confirm & Submit**
Click 'Update' to begin. The control plane update typically takes 10–15 minutes.

5 **Monitor Progress**
Stay on the cluster detail page. The Status will show 'UPDATING'. Wait until it changes to 'ACTIVE'.

WARNING: Do NOT proceed to node group upgrades until the control plane status shows ACTIVE and the Kubernetes version reflects the new version.

Verify Control Plane Upgrade

```
# Confirm control plane is on the new version
aws eks describe-cluster --name <cluster-name> --query 'cluster.version'

# Also visible via kubectl (server version = control plane)
kubectl version --short

# Expected output:
```

```
# Client Version: v1.28.x
# Server Version: v1.29.x   <-- this should now reflect the new version
```

PHASE 2 — NODE GROUP UPGRADE (ONE NODE AT A TIME)

Node Group Upgrade — NoExecute Taint Strategy

This phase upgrades worker nodes one at a time using the NoExecute taint approach. This ensures zero downtime by gracefully evicting pods from each node before it is upgraded or replaced.

> **NOTE:** Repeat the steps in sections 3.1 through 3.5 for each node in the node group, one at a time. Complete all steps for Node 1 before moving to Node 2.

Understanding the NoExecute Taint Approach

A NoExecute taint on a node does two things simultaneously:

- Prevents new pods from being scheduled onto the tainted node (same as NoSchedule).
- Evicts all existing pods that do not have a matching toleration — immediately and gracefully.

This is more forceful than cordon+drain alone and ensures pods are moved before the node is touched.

Step 3.1 — Identify All Nodes

```
# List all nodes with version and status
kubectl get nodes -o wide

# Example output:
# NAME                          STATUS   VERSION   AGE
# ip-10-0-1-101.ec2.internal    Ready    v1.28.x   30d
# ip-10-0-1-102.ec2.internal    Ready    v1.28.x   30d
# ip-10-0-1-103.ec2.internal    Ready    v1.28.x   30d

# Pick one node to upgrade first — note its exact name
```

Step 3.2 — Cordon the Node

Cordoning marks the node as unschedulable. New pods will not be placed here, but existing pods keep running.

```
# Cordon the node (marks it Unschedulable)
kubectl cordon <node-name>

# Example:
kubectl cordon ip-10-0-1-101.ec2.internal
```

```
# Verify — STATUS should show 'Ready,SchedulingDisabled'
kubectl get node <node-name>
```

Step 3.3 — Apply NoExecute Taint to Evict Pods

This is the core of the strategy. The NoExecute taint immediately evicts all pods without a matching toleration, shifting them to other available nodes.

```
# Apply the NoExecute taint — this evicts pods immediately
kubectl taint nodes <node-name> node-upgrade=true:NoExecute

# Example:
kubectl taint nodes ip-10-0-1-101.ec2.internal node-upgrade=true:NoExecute

# Verify pods are evicting / have moved off the node
kubectl get pods -A -o wide | grep <node-name>

# Wait until no application pods remain on this node
# DaemonSet pods will remain — that is expected and correct
```

NOTE: DaemonSet pods (such as kube-proxy, aws-node, log collectors) will remain on the node — this is expected. They run on every node by design and will be handled when the node itself is replaced.

Step 3.4 — Drain the Node (Final Safety Check)

Although the taint has already evicted pods, running drain as well confirms there are no remaining pods and removes DaemonSet-managed pods from consideration.

```
# Drain to confirm clean state (ignore daemonsets, delete local storage)
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --force

# Example:
kubectl drain ip-10-0-1-101.ec2.internal \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --force
```

WARNING: If drain hangs, check for PodDisruptionBudgets blocking eviction: kubectl get pdb -A. You may need to temporarily scale up the affected deployment before draining.

Step 3.5 — Upgrade Node via Console

With the node drained, now trigger the upgrade through the AWS Console for Managed Node Groups. For self-managed nodes, terminate and replace the instance.

Option A — Managed Node Groups (Console)

- Go to EKS Console → Clusters → Your Cluster → Node Groups.
- Select the node group → Click 'Update now'.
- Choose 'Rolling update' strategy → Set Max unavailable to 1.
- Click 'Update' and wait for the node to be replaced with the new AMI.

Option B — Self-Managed Nodes (Terminate & Replace)

```
# Get the instance ID of the node
kubectl get node <node-name> -o jsonpath='{.spec.providerID}'
# Output format: aws:///us-east-1a/i-0abc12345def

# Terminate the instance — ASG will launch a replacement
aws ec2 terminate-instances --instance-ids i-0abc12345def

# Wait for new node to join the cluster
kubectl get nodes --watch
```

Step 3.6 — Verify New Node & Uncordon

```
# Confirm the new node is Running on the new Kubernetes version
kubectl get nodes -o wide

# Remove the NoExecute taint from the new node
kubectl taint nodes <new-node-name> node-upgrade=true:NoExecute-

# Uncordon to allow scheduling (if node not auto-uncordoned)
kubectl uncordon <new-node-name>

# Verify pods are being scheduled back
kubectl get pods -A -o wide | grep <new-node-name>
```

Step 3.7 — Repeat for Remaining Nodes

Once the first node is successfully upgraded and healthy, proceed to the next node. Follow Steps 3.1 through 3.6 exactly for each remaining node.

> **WARNING:** Do not taint or drain the next node until the previous node is fully Ready and accepting pods. Rushing node upgrades is a common cause of capacity shortfalls.

```
# Quick health check before moving to next node
kubectl get nodes                     # All previous nodes should be Ready
kubectl get pods -A | grep -v Running | grep -v Completed   # No unhealthy pods
```

PHASE 3 — ADD-ONS UPGRADE (AWS CONSOLE)

Upgrading EKS Managed Add-ons

Once all nodes are upgraded, update the core EKS managed add-ons to versions compatible with the new Kubernetes version. This is done through the AWS Console.

WARNING: Add-ons must be upgraded AFTER the control plane and all node groups are on the new version. Upgrading add-ons before nodes can cause compatibility issues.

Core Add-ons to Upgrade

Add-on	Purpose	Upgrade Impact
vpc-cni	Pod networking — assigns IPs to pods	Brief pod connectivity interruption possible
kube-proxy	Network rule management on nodes	Minimal — rolling DaemonSet update
coredns	DNS resolution inside cluster	Brief DNS interruption if only 1 replica
aws-ebs-csi-driver	EBS volume provisioning and attachment	No impact to existing volumes

Step 4.1 — Check Compatible Add-on Versions

```
# Find compatible add-on versions for your new Kubernetes version
aws eks describe-addon-versions \
  --kubernetes-version 1.29 \
  --addon-name vpc-cni \
  --query 'addons[].addonVersions[].addonVersion'

# Repeat for each add-on:
# --addon-name kube-proxy
# --addon-name coredns
# --addon-name aws-ebs-csi-driver
```

Step 4.2 — Upgrade Each Add-on via Console

1 **Open the Cluster Add-ons Tab**
EKS Console → Clusters → Your Cluster → Add-ons tab.

2 **Select an Add-on**
Click on the add-on name (e.g., 'vpc-cni'). You will see the current version and available updates.

3 **Click Edit**
Click the 'Edit' button on the add-on detail page.

4 **Select Target Version**
From the Version dropdown, select the latest version compatible with your Kubernetes version. Look for versions marked 'Default'.

5 **Set Conflict Resolution**
Under 'Conflict resolution method', select 'Overwrite' to allow AWS to overwrite any configuration drift.

6 **Save & Monitor**
Click 'Save changes'. Monitor the add-on status until it shows 'Active'. Then proceed to the next add-on.

TIP: Upgrade add-ons one at a time and wait for each to reach Active status before proceeding to the next one.

Step 4.3 — Verify Add-on Health

```
# Check all add-ons are Active
aws eks list-addons --cluster-name <cluster-name>

# Verify add-on pods are running
kubectl get pods -n kube-system

# Specifically check CoreDNS
kubectl get deployment coredns -n kube-system

# Check vpc-cni DaemonSet
kubectl get daemonset aws-node -n kube-system

# Check kube-proxy DaemonSet
kubectl get daemonset kube-proxy -n kube-system
```

PHASE 4 — POST-UPGRADE VALIDATION

Post-Upgrade Health Checks

After all phases are complete, run a comprehensive validation to confirm the cluster is healthy and all workloads are operating correctly.

Final Cluster Validation

```
# 1. Confirm all nodes are on the new version and Ready
kubectl get nodes -o wide

# 2. Confirm no pods in error state
kubectl get pods -A | grep -vE 'Running|Completed'

# 3. Check overall cluster health
kubectl get componentstatuses

# 4. Verify add-on versions
aws eks describe-addon --cluster-name <cluster-name> --addon-name vpc-cni --query
'addon.addonVersion'
aws eks describe-addon --cluster-name <cluster-name> --addon-name kube-proxy --
query 'addon.addonVersion'
aws eks describe-addon --cluster-name <cluster-name> --addon-name coredns --query
'addon.addonVersion'

# 5. Run a smoke test on critical deployments
kubectl rollout status deployment/<critical-deployment> -n <namespace>
```

Upgrade Summary Checklist

#	Task	Status
1	Pre-upgrade checks completed (API deprecations, backups, health)	☐ Done
2	Control plane upgraded via AWS Console	☐ Done
3	Control plane status: ACTIVE, new version confirmed	☐ Done
4	Node 1 — Cordoned, NoExecute taint applied, pods evicted	☐ Done
5	Node 1 — Drained, upgraded via Console or EC2 replacement	☐ Done
6	Node 1 — New node Ready, taint removed, pods rescheduled	☐ Done
7	Nodes 2…N — Repeated same process for each node	☐ Done
8	All nodes on new Kubernetes version	☐ Done
9	vpc-cni add-on upgraded via Console	☐ Done

10	kube-proxy add-on upgraded via Console	☐ Done
11	coredns add-on upgraded via Console	☐ Done
12	aws-ebs-csi-driver add-on upgraded (if in use)	☐ Done
13	All add-ons in Active state	☐ Done
14	Final health check — all pods Running, no errors	☐ Done

TIP: Archive this completed checklist and the cluster backup files in your team's runbook repository for audit purposes.